POT JS 0.8

This is the Javascript API for the Go POT implementation that allows to write and read key-value pairs to the Swarm decentralized storage network.

A POT storage tree structure allows to have updateable key-value maps stored on an immutable, content-addressable network like Swarm. It is called Proximity Order Tree (POT) after a fundamental, new storage technique that is described in a forthcoming paper.

INDEX

I

USE

QUICK START

Drop `pot.wasm`, `potjs.js`, and `wasm_exec.js` in a directory.
Write some html and js using functions **new**(), **put**(), **get**(), **save**(), **restore**().
Check out the function reference and examples.

There is no need for building the project (i.e., no need to install or run Go) to use it.

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

    await pot.ready()
    map = await pot.new()
    await map.put("hello", "POT!")
    value = await map.get("hello")
    console.log("key ‹hello› has:", value)
})()

</script>
```

INSTRUCTIONS

The JS API is for embedding POT functionality into a browser environment or to run it with node.js.

To initialize, include the generic **wasm_exec.js** and **potjs.js**, then wait for the initialization to finish:

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

        await pot.ready()

        …
})()
…
```

After **pot.ready()** resolves, the Go WASM **pot** object can be used to create a new map. Use **put** and **get** functions on it.

```
map = pot.new()

await pot.put("hello", "Hello, P.O.T.!")

value = await pot.get("hello")
```

See **example1.html**.

You may leave out **potjs.js** to initialize with more direct control: include only the generic Go WASM wrapper `wasm_exec.js`, create a `Go` object and instantiate the POT WASM object **pot.wasm**, then run it once it is loaded. See **example3.html**:

```
<script src="wasm_exec.js"></script>

<script>

const go = new Go()
WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
        .then((potwasm)=>{ go.run(potwasm.instance) })
…
```

FUNCTION OVERVIEW

The focus of the functions are **get** and **put**. They are split in two by two main groups, for four different ways of calling. Additionally, there are init, test and support functions.

The groups are asynchronous / synchronous and typed / raw respectively. Asynchronous functions return promises, synchronous functions block and return a value, error or null. Typed functions mark the Javascript type in the storage, to automatically return the right type on retrieval. This can be boolean, number,[1] or string. Raw functions accept a Uint8Array of bytes to store and the right function has to be called to get the expected type back out. The synchronous functions' names end on Sync. The raw functions have Raw as a name component. The most relevant functions are put() and get(), which are asynchronous, typed functions. That means, get() returns a promise for the value to be retrieved and it will automatically be in the type that it was stored with put().

Synchronous functions can be handy for prototyping and special situations but generally only work for tests in the browser using the in-memory persistence of the Go POT implementation, not when connecting to Swarm or a test network, and not when using node. It is possible that they might work well with the background threads of Web Workers but this has not been tested. Therefore, they should simply NOT be used in production. The asynchronous functions work in all circumstances.

Raw functions will be useful in special situations, whenever binary data is handled, but usually the vanilla (typed, asynchronous) functions **new**, **save**, **put** and **get** will be the first choice.

For details and examples, see file **Pot – (III) 31 August 2025 function reference.pdf.**

MAP INITIALIZATION

pot.new	create a new Swarm key-value-store, i.e., a map, async
pot.newByReference	create a new Swarm map from a reference of a prior save, async
map.save	store a Swarm KVS to the network, async
pot.newSync	create a new Swarm key-value-store, i.e., a map, sync
pot.newByReferenceSync	create a new Swarm map from a reference of a prior save, sync
map.saveSync	store a Swarm KVS to the network, sync

TYPE-AWARE

put	return a promise for null or Error. Asynchronous.
get	return a promise for the typed get, i.e., the type will be as put. Async.

[1] Javascript does not have native integers and stores all numbers as IEEE 754 64-bit floats internally.

`putSync`	return a promise for null or Error. Synchronous.
`getSync`	return a promise for the typed get, i.e., the type will be as put. Sync.

RAW BYTE ACCESS

`putRaw`	put a raw `Uint8Bytes` array, async.
`getRaw`	get a raw `Uint8Bytes` array, async.
`getBoolean`	get the raw value as Boolean. First byte 0 for false, all else true, async.
`getNumber`	get the raw value as floating point number. Must be 8 bytes. Async.
`getString`	get the raw value as string, async.
`putRawSync`	put a raw `Uint8Bytes` array, synchronous.
`getRawSync`	get a raw `Uint8Bytes` array, synchronous.
`getBooleanSync`	get the raw value as Boolean. First byte 0 for false, all else true, sync.
`getNumberSync`	get the raw value as floating point number. Must be 8 bytes. Sync.
`getStringSync`	get the raw value as string, synchronous.

POT INITIALIZATION

`pot.ready`	promise that resolves when pot.wasm is initialized. Include `potjs.js`.
`onWasmLoaded`	called, if it exists, by pot.wasm when it is initialized. Without `potjs.js`.

TESTING AND SUPPORT

These functions are not used in production. They serve to test the integrity of POT JS.

`log`	write to browser console via Go to verify a language-land roundtrip
`hello`	write hello to browser console to verify established WASM stream
`randKey`	32 random bytes for key testing
`randValue`	79 to 101 random bytes for value testing (range from Go POT)
`type_encoded_bytes`	JS type detection and value encoding with correct leading type byte
`type_decoded_value`	JS type detection of raw kvs value by leading type byte
`hangingPromise`	blocking promise (in Go) with time out for exception testing

`setFail`	trigger the next mock storage function called to return an error
`setPanic`	trigger the next mock storage function called to raise a panic
`setDelay`	trigger the next mock storage function called to stall for a given time
`setHang`	trigger the next mock storage function called to stall indefinitely

EXAMPLES

To run the examples, serve the main directory with `npx http-server .` and open `http://127.0.0.1:8080/example1.html`.

Or use

```
make example1
```

etc. There are seven examples 1-7 that can be started this way.

`example1.html`	briefest example as listed above, in-memory storage
`example2.html`	interactive example, input fields, integrity
`example3.html`	example 2 dropping potjs.js for more control
`example4.html`	example 1 using synchronous access functions
`example5.html`	example 1 but with connection to a storage network
`example6.html`	example 5 with better connection to make rules
`example7.js`	example 1 for execution with node.js

EXAMPLE 1

A simple put and get, logging to console.

example1.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

      await pot.ready()
      map = await pot.new()
      await map.put("hello", "POT!")
      value = await map.get("hello")

      console.log("key ‹hello› has:", value)
})()

</script>
```

WHAT YOU SHOULD SEE

Browser console:

```
pot:  » POTWASM
wasm_exec.js:22 pot:  » init done
wasm_exec.js:22 pot:  » ready
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 1
wasm_exec.js:22 pot:  » put hello: POT!
wasm_exec.js:22 pot:  » →  68656c6c6f0000000000000000000000000000000000000000000000000000:
03504f5421
wasm_exec.js:22 pot:  » get hello: POT!
wasm_exec.js:22 pot:  » ←  68656c6c6f0000000000000000000000000000000000000000000000000000:
03504f5421
example1.html:29 key ‹hello› has: POT!
```

EXAMPLE 2: INTERACTION

Interactive use and integrity checks.

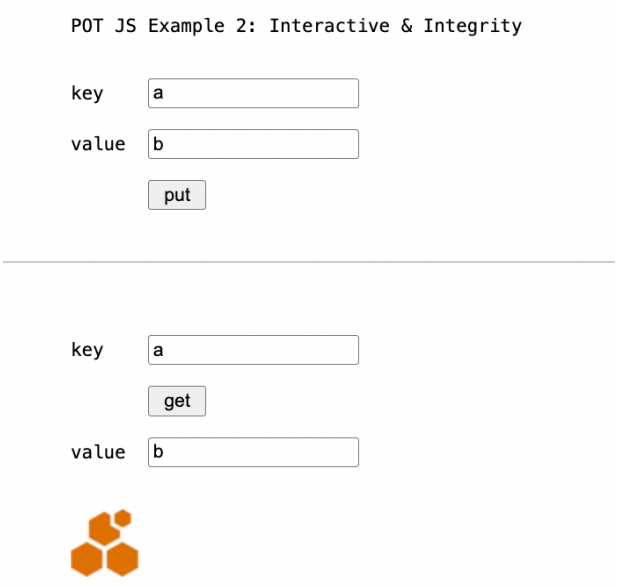
example2.html

```
<pre>

      POT JS Example 2: Interactive & Integrity

      key    <input id=key>
      value  <input id=value>
             <button onclick="put()"> put </button>

<hr />

      key    <input id=search>
             <button onclick="get()"> get </button>
      value  <input id=result>

      <img src=favicon.ico>

</pre>

<script src="wasm_exec.js" integrity="sha384-PWCs+V4BDf9yY1yjkD/p+9xNEs4iE-
buvq+HezAOJiY3XL5GI6VyJXMsvnjiwNbce"></script>
```

```
<script src="potjs.js" integrity="sha384-dUfhLIycF6GUprYxIlI-
iAAQR3rh3KsPxPdsgq4tP2rTe7grND7bXjAj4nawjq5Ht"></script>

<script>

	var map

	(async () => {

		await pot.ready()
		map = await pot.new()

	})()

	const field = (id) => { return document.getElementById(id) }

	async function put() {

		key = field('key').value
		value = field('value').value
		await map.put(key, value)
	}

	async function get() {

		key = field('search').value
		value = await map.get(key)
		field('result').value = value
	}

</script>
```

WHAT YOU SHOULD SEE

After entering a value for key and value, and hitting put. Then entering the same key in the lower screen area and hitting get should yield the value in the lowest field.



Browser Console:

```
pot:  » POTWASM
wasm_exec.js:22 pot:  » init done
wasm_exec.js:22 pot:  » ready
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 1
wasm_exec.js:22 pot:  » put a: b
wasm_exec.js:22 pot:  » ⟶  61000000000000000000000000000000000000000000000000000000000000000000000000:
0362
wasm_exec.js:22 pot:  » get a: b
wasm_exec.js:22 pot:  » ⟵  61000000000000000000000000000000000000000000000000000000000000000000000000:
0362
```

EXAMPLE 3: INITIALIZATION

More explicit initialization.

This page does not use pot.js but initializes pot.wasm directly.

example3.html

```
<pre>

      POT JS Example 3: Explicit Initialization

      key    <input id=key>
      value  <input id=value>
             <button onclick="put()"> put </button>

<hr />

      key    <input id=search>
             <button onclick="get()"> get </button>
      value  <input id=result>

</pre>
<script src="wasm_exec.js"></script>
<script>
      const go = new Go()
      var map;
      WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
            .then((r)=>{
```

```
                go.run(r.instance)
                map = pot.newSync()
        })

    const field = (id) => { return document.getElementById(id) }

    async function put() {

        key = field('key').value
        value = field('value').value
        await map.putSync(key, value)
    }

    async function get() {

        key = field('search').value
        value = await map.getSync(key)
        field('result').value = value
    }

</script>
```

EXAMPLE 4: SYNCHRONOUS

This page uses synchronous calls that can be helpful for prototyping but don't work for network connections (only in-memory = new() without parameters) and not for use with node.

example4.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

function onWasmLoaded() {

    map = pot.newSync()

    map.putSync("hello", "POT!")

    value = map.getSync("hello")

    console.log("key ‹hello› has:", value)
}

</script>
```

EXAMPLE 5: NETWORK

This page connects to a local Swarm network.

The example is for illustration, the batch id would have to be updated. See example5.html (and 6).

example5.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

      await pot.ready()

      map = await pot.new("http://localhost:1633",
      "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa")

      await map.put("hello", "POT!")

      value = await map.get("hello")

      console.log("key ‹hello› has:", value)

      ref = await map.save()

      console.log("tree root is:", ref)

})()

</script>
```

EXAMPLE 6: NETWORK II

Example 6 works like example 5 but can be run directly with `make example6`.

See **example6.html**

EXAMPLE 7: NODE

example7.js

.

```
fs = require("fs");
require("./wasm_exec"); // note the ./

var go = new Go();

WebAssembly.instantiate(fs.readFileSync("pot.wasm"), go.importObject)
      .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {

      console.log("wasm loaded-function called")

      map = await global.pot.new()

      await map.put("hello", "node")

      value = await map.get("hello")

      console.log("hello:", value)
}
```

WHAT YOU SHOULD SEE

Terminal:

```
      POT JS Example 7: Node

      Check out the source in example7.js.


pot:  » POTWASM
pot:  » init done
wasm loaded-function called
pot:  » initialize new P.O.T.
pot:  slot ref: 1
pot:  » ready
pot:  » put hello: node
pot:  » ⟶  68656c6c6f0000000000000000000000000000000000000000000000000000: 036e6f6465
pot:  » get hello: node
pot:  » ⟵  68656c6c6f0000000000000000000000000000000000000000000000000000: 036e6f6465
hello: node
```

II

BUILDING

There is **no need for building POT JS to use it.** The main executable, the Go WASM file `pot.wasm`, and the auxiliary Javascript and HTML files are portable.

To build, run

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
```

or use

```
make build
```

WHAT YOU SHOULD SEE

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
```

CLEAN BUILD

To rebuild from scratch, including integrity hashes and rebuilding `go.mod`, use:

```
make clean build
```

WHAT YOU SHOULD SEE

```
rm -f go.mod
rm -f pot.wasm
rm -f wasm_exec.js
rm -f potjs.js.sha384 wasm_exec.js.sha384
rm -f .batch_creation .batch_id
go mod init potwasm
go: creating new go.mod: module potwasm
go: to add module requirements and sums:
        go mod tidy
go mod edit -replace github.com/ethersphere/proximity-order-
trie=github.com/ethersphere/proximity-order-trie@v1.0.0
go get
go: added github.com/beorn7/perks v1.0.1
go: added github.com/cespare/xxhash/v2 v2.2.0
go: added github.com/ethersphere/bee/v2 v2.5.0
go: added github.com/ethersphere/proximity-order-trie v1.0.0
go: added github.com/hashicorp/errwrap v1.0.0
go: added github.com/hashicorp/go-multierror v1.1.1
go: added github.com/prometheus/client_golang v1.18.0
go: added github.com/prometheus/client_model v0.6.0
go: added github.com/prometheus/common v0.47.0
go: added github.com/prometheus/procfs v0.12.0
go: added golang.org/x/crypto v0.23.0
go: added golang.org/x/sys v0.20.0
go: added google.golang.org/protobuf v1.33.0
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
openssl dgst -sha384 -binary potjs.js | openssl base64 -A > potjs.js.sha384
openssl dgst -sha384 -binary wasm_exec.js | openssl base64 -A >
wasm_exec.js.sha384
for fn in potjs.js.sha384 wasm_exec.js.sha384; do sed -i.bak -e "s/\(\"${fn:0:-7}\"
integrity=\)[^>]*/\1\"sha384-${$(cat ${fn})//[\/]/\\/}\"/" example1.html ; rm -f
example1.html.bak ; done
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
```

REQUIREMENTS

POT JS includes the Go implementation of POT.

POT JS API

go 1.24.0

GO POT

go 1.24.0

testify 1.8.4
swarm bee 2.5.0
crypto 0.23.0
sync 0.14.0

Which will require

perks 1.0.1
xxhash 2.2.0
go-spew 1.1.1
errwrap 1.0.0
go-multierror 1.1.1
text v0.2.0
go-difflib 1.0.0
prometheus
x/sys 0.20.0
protobuf 1.33.0
yaml 3.0.1

See **go.mod**.

TESTS

**To test the JS API, serve the main directory, e.g., with with npx http-server .
and open http://127.0.0.1:8080/test.html?tag=in-mem.**

Or use

```
make test
```

It uses `test.js` and `test.css` to display the results from a suite of 142 tests across the available `put` and `get` functions, and more.

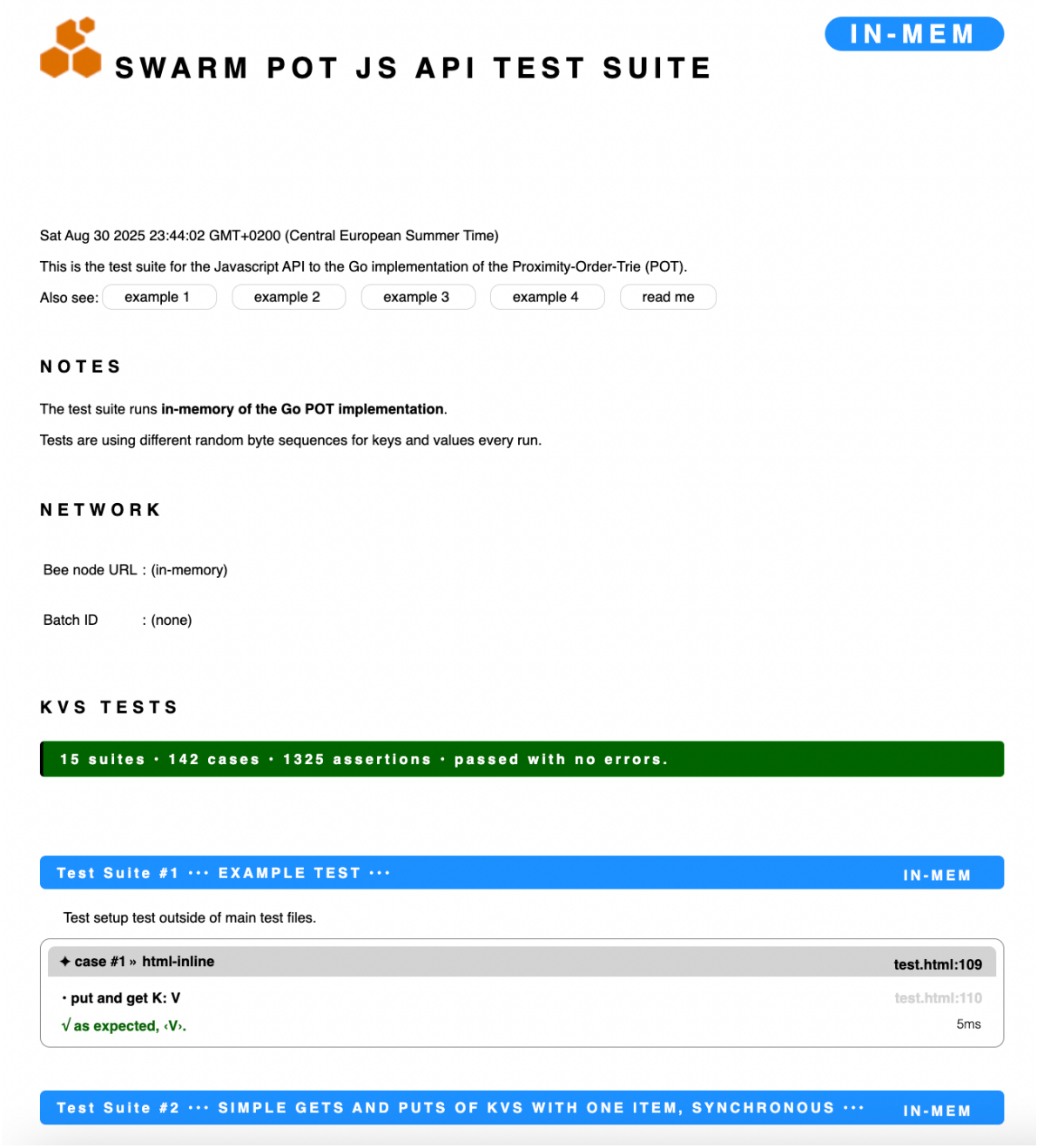
WHAT YOU SHOULD SEE

Terminal

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
⬡ start http server (stop with 'make stop')
(npx http-server -c1 . &)
open "http://127.0.0.1:8080/test.html?tag=in-mem"
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

[2025-08-30T21:44:02.883Z]  "GET /test.html?tag=in-mem" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:22323) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-08-30T21:44:02.904Z]  "GET /test.css" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.906Z]  "GET /wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

```
[2025-08-30T21:44:02.910Z]  "GET /favicon.ico" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.989Z]  "GET /test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.990Z]  "GET /kvs_test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.997Z]  "GET /pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

Browser



Browser Console

...

```
wasm_exec.js:22 pot:  ✦ case #140 » Cancel Delay
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  create hanging test promise and cancel it (delay, chain .catch)
wasm_exec.js:22 pot:  sleeping
wasm_exec.js:22 pot:  canceled!
wasm_exec.js:22 pot:  √ expected error: ‹Error: canceled›
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ✦ case #141 » Immediate Cancel II
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  create hanging test promise and cancel it immediately (chain .catch)
wasm_exec.js:22 pot:  sleeping
wasm_exec.js:22 pot:  canceled!
wasm_exec.js:22 pot:  √ expected error: ‹Error: canceled›
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• Test Suite #14 ••• FAILURES •••▶IN-MEM◀
wasm_exec.js:22 pot:  Failure tests are available in extended API test mode. Run `$make xt`.
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• Test Suite #15 ••• PROOFS •••▶IN-MEM◀
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ✦ case #142 » Return a Proof
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  • new map
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 101
wasm_exec.js:22 pot:  √ no error raised.
wasm_exec.js:22 pot:  • get proof for K1
wasm_exec.js:22 pot:  √ as expected, ‹null›.
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• tests cases run: 142 • assertions: 1325  •••
wasm_exec.js:22 pot:  ▶ no errors ◀
```

The standard tests run against the Go implementation of POT, like the use would be in production. The **extended tests**, however, focus on the JS AP to Go, add cancellation and time out cases. They do not interface with the Go POT code.

Latency emulation, promises, and connectivity as well as retention error conditions will be added to emulate the reality of an unreliable network as ultimate storage layer. Run the tests with `make test` and `make xt` respectively.

Note that `make xt` (specifically, `make mock` change the project configuration and build executables (especially a `pot.wasm`) that will not work for production. (They will appear to work but they sidestep Go POT.) Use `make unmock`, `make test` or `make build` to restore to the production target (i.e., build the real `pot.wasm`).

The **extended tests** use a separate, dedicated, in-memory mock storage to separate them from the Go POT implementation.

The tests are made for the browser as the final destination of the combined architecture, in either case (standard and extended tests) they are invoked by opening the file `test.html`. The very small file that is altered between the test modes is `testmode.js`, and `go.mod` to replace package the real Go POT with `mock/`

This test page and test framework is designed to bridge the language divide and allow for the emulation of exceptions in one degree of separation (caused in Go, caught from JS). The

major relevance of the test harness is to help surface any friction between JS's and Go's resource release mechanisms (JS promises and Go channels) to detect possible leaks. Go's channels are not without peril — they can deadlock — and the challenge is aggravated by their position behind the JS API. The relevant code triggering the exceptional conditions resides in `mock.go`. For its purposes, it has to be part of the `main` package rather than package `pot` in `mock/`.

III

DEVELOPING

Developing and testing at this point have been done on Mac only.

The main tests are browser-based.

FILES

```
README.MD             subset of this document

potjs.go              Go JS interface
nomock.go             No op version of test functions
pot.wasm              Go POT implementation compiled to WASM
potjs.js              Thin JS initialization convenience code

wasm_exec.js.sha384   SHA384 checksum for sub-component verification
potjs.js.sha384       SHA384 checksum for sub-component verification

example1.html         briefest example as listed above, in-memory storage
example2.html         interactive example, input fields, integrity
example3.html         example 2 dropping potjs.js for more control
example4.html         example 1 using synchronous access functions
example5.html         example 1 but with connection to a storage network
example6.html         example 5 with better connection to make rules
example7.js           example 1 for execution with node.js

wasm_exec.js          generic Go WASM wrapper (replace w/your Go version's)

test.html             test main page for test, ext_test, locnet_test
kvs_test.js           test suites
test.js               generic test harness
test.css              generic test optics
favicon.ico           Swarm logo to suppress browser errors

mock.go               monkey patch mock of Go POT for 'extended' testing
mock/pot.go
mock/go.mod
mock/pkg/persister    copy from Go POT in-memory storage (not used)

go.mod                go module locations
go.sum                go module check sums

Makefile              build scripts
```

MAKE

The project comes with an extensive `Makefile` to support developing.

Below are the build rules for executables and for running tests of POT JS.

Note that building and any of the following is not required for using POT JS. The relevant executable files are portable and part of the repo. After cloning or downloading a distribution of POT JS, all relevant files are immediately usable.

RULES

Run with `make <rule>`

Building & Testing

`build`	build executables
`example<n>`	start server and run example <n>. n = 1-7
`test`	test api interaction with go pot in-memory persisting
`ext_test`	extended exceptions tests without go pot connection
`locnet_test`	test with a locally installed Swarm network
`clean`	prepare for building from scratch
`distclean`	prepare for commit to repository, including build

Support & Debugging

For day-to-day developing, these commands may be helpful.

`serve`	start local http server serving current directory
`stop`	stop local http server. Ctrl-c does not.
`mock`	prepare for extended exception tests
`unmock`	reset for production or non-extended tests
`locnet_install`	install the local test network
`locnet_start`	start the local test network on docker
`locnet_batch`	buy stamp batch (free)
`locnet_tests`	run test suites on local test network
`locnet_stop`	shut local test network down
`locnet_logs`	show the entire log of the queen node

DEVELOPER NOTES

ERROR STRATEGY

The error strategy of the API is JS-Style, using JS exceptions triggered from the go wasm functions via promise mechanisms, rather than returning error codes, Go-style.

The implementation of this is in Go though (`potjs.go`), not in JS, creating JS code in Go, with full data/situational access on the Go side.

NOTES ON GO POT IN WASM

Being wrapped in WASM for the interfacing with JS, the Go executable is opaque, it cannot be debugged as there are no debuggers for wasm, and it becomes unavailable once it has panicked.

NOTE ON SYSCALL/JS

The connection between Go and Javascript through WASM is based on Go's syscall/js package. This package is still officially labeled as experimental:

> "This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise."
>
> https://pkg.go.dev/syscall

IV

ROADMAP

Possible future improvements:

- **PROMISE CANCELLATION FUNCTIONS**
- **FUNCTION TIMEOUTS**
- **PROOF RETRIEVAL FUNCTION**
- **TREE WALK FUNCTIONS**
- **PROJECT FOLDER REORGANIZATION**
- **GO-SIDE RESOURCE TRACKING INSTRUMENTATION**
- **NODE TESTS**
- **INTEGRATION TESTS**
- **JS-NATIVE PERSISTERS**
- **USE OF JS NEW OPERATOR**